

30.09.2024



## CSE 301 ALGORITHMS LAB 2

### Job Application

You're running an HR department for job applications. You need a simple application program that sorts applicants by their experience level. If the applicants have the same experience level. We know their experience level by **the number of years**. You also consider the **minutes to commute to work** for the applicants. You do not provide transportation service, so you prefer applicants close to you.

**There needs to be two levels of sorting.** The first level of sorting is ordering the applicants by their number of experiences in ascending order. The second level of sorting is ordering the applicants according to their commute time in descending order.

Implement the following functionality using a combination of insertion sort and merge sort.

- Use merge sort to sort the overall list by experience, then by commute time.
- Use insertion sort to sort smaller chunks of the data by experience, then by commute time. **you need to choose the size of the chunks.**

As you know, merge sort is a recursive algorithm. Recursive algorithms have base cases. A base case will be applied when the number of elements exceeds 1 and less than your **chosen chunk size**.

Make sure your work has a reasonable runtime. The runtime might be affected by your chunk size at larger inputs.

## Inputs:

[(2, 100), (5, 50), (3, 20), (5, 120), (2, 40)]

The first applicant has 2 years of experience, and it takes 100 minutes to commute to work. The second applicant has 5 years of experience, and it takes 50 minutes to commute to work.

## Output:

[(5, 50), (5, 120), (3, 20), (2, 40), (2, 100)]

## Constraints:

You must use both insertion sort and merge sort in your solution. The maximum number of applications n is 1000.

## Answer:

```
1 public class JobApplicationSort {
2
3     // Function to perform merge sort on the job
4     // applications
5     public static void primarySort(int[][] applications,
6                                     int left, int right) {
7         // codes here
8     }
9
10    // Insertion sort for small subarrays
11    public static void insertionSort(int[][] applications,
12                                     int left, int right) {
13        // codes here
14    }
15
16    // Final Outer Merge function {You might need}
17    public static void merge(int[][] applications, int left
18                             , int middle, int right) {
19        // codes here
20    }
```

```

18     public static void main(String[] args) {
19         int[][] applications = {
20             [(2, 100), (5, 50), (3, 20), (5, 120), (2,
21                 40)]
22         };
23
24         System.out.println("Original applications:");
25         printApplications(applications);
26
27         primarySort(applications, 0, applications.length -
28             1);
29
30         System.out.println("Sorted applications by
31             experience and order:");
32         printApplications(applications);
33     }
34
35     // Helper function to print applications
36     public static void printApplications(int[][]
37         applications) {
38         for (int[] app : applications) {
39             System.out.println("Experience: " + app[0] + ",
40                 Order: " + app[1]);
41         }
42     }
43 }

```